1.- A, B, C, E y F. Tipos no permitidos:

- D. long y H. double: No están permitidos en un switch tradicional (al igual que float y boolean).
- G. var: No es un tipo de datos en sí, sino una palabra reservada para la inferencia de tipos de variables locales

2.- B. Prueba de escritorio.
3.- A, C, F y H. En Java, el lado derecho de un bucle for-each (for (var item : expression)) debe ser un array o una instancia de java.lang.Iterable.
4.- F:

• Exhaustividad: En Java, una expresión `switch` (asignada a una variable como `type`) debe gestionar todos los valores posibles del tipo de entrada (para un `int`, esto significa abarcar todos los valores enteros o proporcionar un caso predeterminado).
• Falta de caso predeterminado: Dado que los casos solo cubren los valores del 1 al 8, cualquier otro valor entero pasado a `category` no sería gestionado, lo que provocaría un error de compilación.

5.- E. El código no compila debido a una línea inalcanzable.
6.- C, D y E.

• En un bucle for tradicional, la condición de parada se evalúa antes de la primera ejecución del cuerpo del bucle. Si es falsa desde el inicio, el bucle no se ejecuta.
En las expresiones switch (introducidas en versiones recientes de Java que devuelven un valor y lo asignan a una variable), el compilador exige la exhaustividad. Dado que un objeto String puede tener infinitos valores posibles, se requiere obligatoriamente una rama default para cubrir todos los casos no especificados explícitamente.
El bucle do/while evalúa la condición al final de la iteración. Por lo tanto, el cuerpo del bucle siempre se ejecuta al menos una vez, independientemente de si la condición es verdadera o falsa.

7.- B. Inicializa i = 0, comprueba i <= weather.length - 1 (equivalente a i < weather.length) e incrementa con ++i, accediendo correctamente a cada elemento desde el índice 0 hasta weather.length - 1 en orden.
8.- G. Porque la línea 36 no compila (debido a las reglas de alcance de flujo con el operador ||, donde no se garantiza que `bat` esté definido si no es un `Long`) y la línea 38 no compila porque `default` no puede utilizarse con una sentencia `if-else`.
9.- B. Insertar `break RABBIT` o `break` en el espacio en blanco finaliza el bucle interno (RABBIT) en la primera iteración en la que se incrementa `count`; esto significa que `count` se incrementa una vez por cada iteración del bucle externo (BUNNY), alcanzando un total de 2 antes de que el bucle finalice o se interrumpa prematuramente.
10.- E. Recordar las propiedades de los bucles.
11.- A.

• La expresión `switch` evalúa el valor de enumeración `Animal.MAMMAL`.
• La rama `case MAMMAL -> 3;` coincide con la entrada.
• El valor 3 se asigna a la variable `type` y se imprime mediante `System.out.print(type)`.

12.- C.

Primera iteración:

-> Se evalúa la condición del bucle while(sing > squawk) \(\rightarrow \) 8 > 2 es true.

->sing-- (la separación de línea de la instrucción ; en la línea 5 es válida en Java) \(\rightarrow \) sing pasa a ser 7.

->squawk += 2 \(\rightarrow \) squawk pasa a ser 4.

->notes += sing + squawk \(\rightarrow \) notes = 0 + 7 + 4 \(\rightarrow \) notes ahora vale 11.

Segunda iteración:

->Se evalúa la condición while(sing > squawk) \(\rightarrow \) 7 > 4 es true.

->sing-- \(\rightarrow \) sing pasa a ser 6.

->squawk += 2 \(\rightarrow \) squawk pasa a ser 6.

->notes += sing + squawk \(\rightarrow \) notes = 11 + 6 + 6 \(\rightarrow \) notes ahora vale 23.

Tercera comprobación:

->Se evalúa la condición while(sing > squawk) \(\rightarrow \) 6 > 6 es false.

El bucle termina.

13.- G. El código no compila debido a la línea 8. En Java, la condición de un bucle `do-while` siempre debe ir entre paréntesis.

-> Sintaxis incorrecta (línea 8): `} while keepGoing;`

-> Sintaxis correcta: `} while (keepGoing);`

Si los paréntesis estuvieran presentes, el bucle se ejecutaría dos veces (cuando `meters` pasa a ser 9 y luego 8), restando 2 a `result` en cada iteración, lo que daría como resultado 11 (opción D). Sin embargo, debido a la ausencia de paréntesis, se produce un error de compilación.

14.- B, D Y F.

- B. El tipo de datos de penguin es int. (El array new int[2] almacena elementos primitivos de tipo int, por lo que var infiere int para penguin).
- D. El tipo de datos de emu es Character. (El array ostrich se declara como Character[], por lo que var infiere Character para emu).
- F. El tipo de datos de macaw es Integer. (La lista parrots está tipada explícitamente como List<Integer>, por lo que iterar sobre ella hace que var infiera Integer para macaw).

15.- E y F.

Sintaxis inválida en las etiquetas case: La línea case 'B': 'C': es sintácticamente incorrecta en Java. No se pueden encadenar dos literales separados por dos puntos de esa manera en un switch tradicional. Para agrupar casos en versiones clásicas de Java se debe escribir case 'B': case 'C':.

Uso de variables en las etiquetas case: Aunque a y e están declaradas como final, Java requiere que los valores en los case sean expresiones constantes determinadas en tiempo de compilación. Dependiendo de dónde y cómo estén declaradas estas variables (por ejemplo, si no se consideran constantes en el alcance del método), el compilador arrojará un error indicando que se requiere una expresión constante.

16.- A, B y D. Prueba de escritorio.

17.- B y E. Prueba de escritorio.

18.- C, D y E.

• C. La coincidencia de patrones con una sentencia `if` se implementa utilizando el operador `instanceof`.

• D. No se puede acceder a la variable de patrón después de la sentencia `if` en la que se declara. (Su ámbito se limita a las partes del código donde el compilador sabe que la coincidencia se ha producido con éxito).

• E. El alcance basado en el flujo (*flow scoping*) implica que una variable de patrón solo es accesible si el compilador puede determinar su tipo.

19.- F.- El código compila pero produce un error en tiempo de ejecución.

20.- A y E. El bucle más interno L3: `for( ; ; )` es un bucle infinito porque carece de una condición de terminación. Para evitar un bucle infinito, la línea 12 debe contener una instrucción que interrumpa L3 o dirija la ejecución fuera de él.

21.- E. Prueba de escritorio.

22.- F. El código falla en la compilación debido a las reglas de la estructura switch en Java.

23.- F.

• La línea 19 comienza con una sentencia `else` (`else if`), pero queda huérfana porque un `else` debe seguir inmediatamente a un bloque `if`, y la línea 18 (`else`) ya ha cerrado la cadena condicional.

• No existe una sentencia `if` previa con la que la línea 19 pueda corresponderse, lo que provoca un error de compilación.

• Cabe destacar que la línea 17 (`if (older = true)`) compila correctamente porque la asignación de una expresión booleana dentro de una condición `if` es válida en Java (aunque asigna `true` a `older` en lugar de realizar una comparación).

24.-G.

• El código utiliza la palabra clave `in` dentro del bucle `for` mejorado (`for(var friend in friends)`).

• Java utiliza dos puntos (`:`) para los bucles `for` mejorados (p. ej., `for(var friend : friends)`), no la palabra clave `in` (que se utiliza en lenguajes como JavaScript, Python o C#).

• Debido a la sintaxis no válida (`in`), el código no se compilará independientemente del tipo de datos elegido para `friends`.

25.- D.

1. Evaluación de la variable: La variable instrument tiene el valor "violin".
2. Evaluación del switch: El programa busca un case que coincida exactamente con "violin".
    - case "bass" no coincide.
    - case CELLO (que es "cello") no coincide.
    - case "VIOLIN" no coincide (Java distingue entre mayúsculas y minúsculas, por lo que "violin" no es igual a "VIOLIN").
    - No hay ningún case "violin".
3. Bloque default: Al no haber ninguna coincidencia, la ejecución salta directamente a la línea 13 (default:).
4. Efecto de caída (Fall-through): Como el bloque default y los siguientes case no tienen la instrucción break, el código continúa ejecutándose hacia abajo de forma consecutiva.

Finalmente, System.out.print(p) imprime el valor actual de p, que es 2.

26.- F. El código compila, pero nunca termina durante la ejecución.

27.- F. Debido a esta ruta de rendimiento faltante en el caso 2.
28.- F. Puesto que el código no compila debido a la declaración duplicada de la variable de patrón `guppy`, ninguna de las opciones de ejecución en tiempo de ejecución (A, B, C, D, E) es correcta.
29.- C. Prueba de escritorio (-1 0 1 2 3 4 5 6)